

SIH26249 — 03 Architecture & Tech Stack

SIH 2026 — SIH26249

Air Power: Predictive Maintenance & Fleet Availability

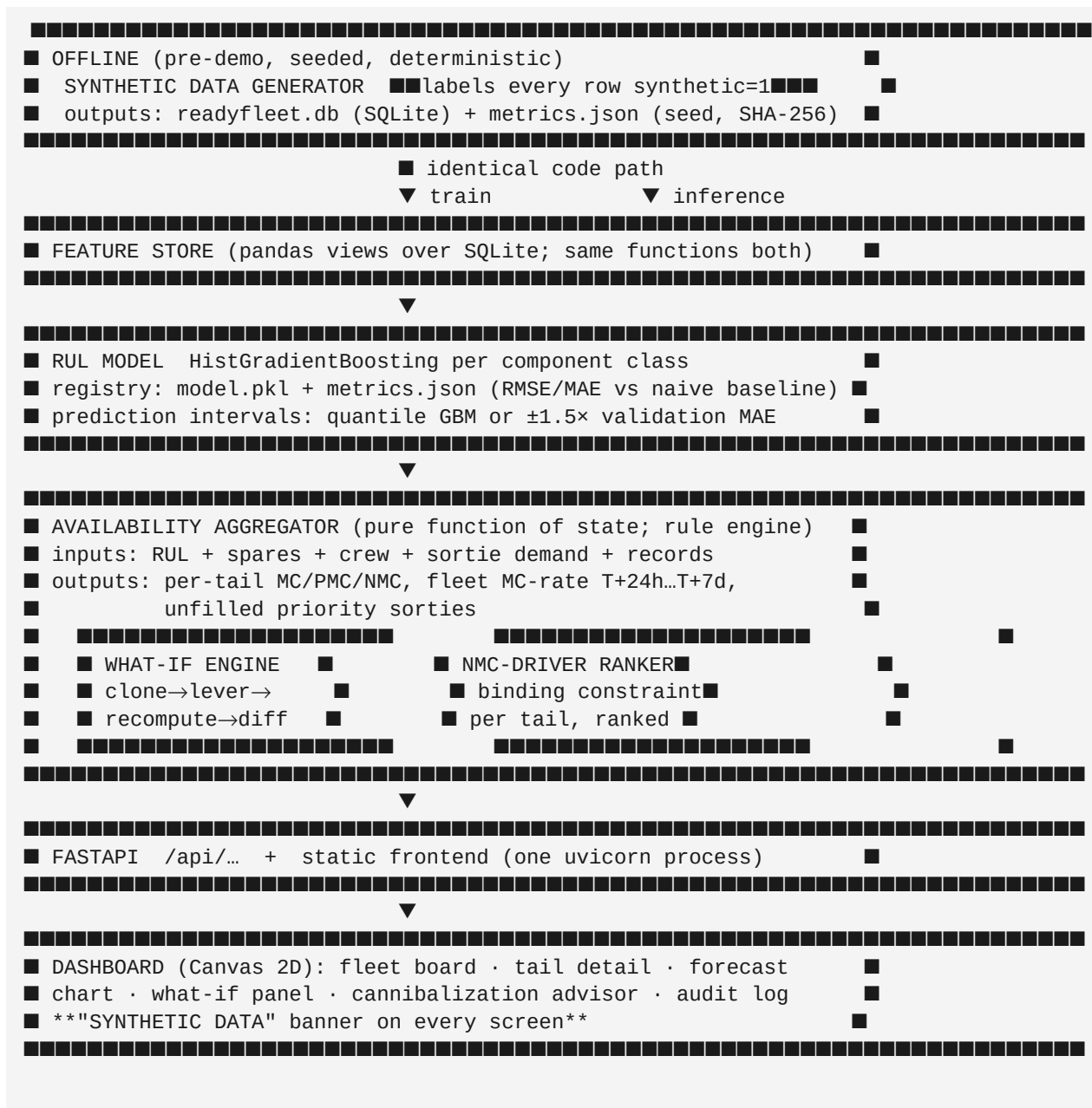
Ministry of Defence — Defence Services Staff College

03 — Architecture & Tech Stack

Stack table

Layer	Choice	Rationale
Frontend	Vanilla JS + Canvas 2D SPA	Team's proven 60 Hz Canvas work; fleet board, cards, heatmaps, gauges all Canvas 2D — zero framework build cost in 36 h
Map	MapLibre GL, optional secondary tab only	Geographic view is secondary; keep it out of the critical path
Backend	Single Python FastAPI service (serves static frontend too; one uvicorn process)	One deployable; Pydantic validation; auto OpenAPI docs
ML	Scikit-learn HistGradientBoostingRegressor for RUL	Trains in seconds on CPU; GBM-on-windowed-stats matches the published LSTM accuracy band on FD001 (~13–20 vs ~12–18 RMSE) while debuggable and live-retrainable. LSTM/Transformer explicitly NOT 36h-feasible
Simulation	Python batch NumPy/Pandas generator, seeded, pre-bakes full synthetic dataset before demo; tiny JS ticker replays sensor values for "live" feel (stated openly)	Deterministic; no live-compute surprises during judging
Database	SQLite single file (fleet ~40 tails is trivial)	Zero-ops; Postgres swap path documented for future work
Deployment	Laptop-local uvicorn (Vercel backup only)	No cloud dependency during judging
Testing	pytest, incl. a model-eval script whose printout IS the judge evidence	Evidence, not just green tests

Architecture diagram (text)



Component responsibilities

Component	Owns	Must NOT
Generator (<code>gen/</code>)	Seeded synthetic fleet, sensors, failures, maintenance history, spares, crew, sorties; writes DB; records seed + SHA-256 in <code>metrics.json</code>	Touch the model or the API
Feature store (<code>features/</code>)	Windowed sensor stats, one-hots, labels; one code path used by train and inference	Leak future cycles into features (validation split BY TAIL)

Component	Owns	Must NOT
RUL model (<code>models/</code>)	Train/eval per component class; model registry; intervals	Claim transfer to real aircraft
Aggregator (<code>core/aggregator.py</code>)	Pure function state→{tail states, MC forecast, unfilled sorties}; all thresholds in <code>config/thresholds.yaml</code>	Hold mutable global state
What-if engine (<code>core/whatif.py</code>)	Clone snapshot → apply lever patch → recompute → diff → store <code>scenario</code> row	Mutate the base state
Driver ranker (<code>core/drivers.py</code>)	Binding constraint per NMC tail; fleet-impact score; top-5	—
Cannibalization advisor (<code>core/cannibal.py</code>)	Donor proposal with guardrails; debt-ledger entries	Propose equal/higher-priority or MC donors (unit-tested)
API (<code>api/</code>)	FastAPI routes, Pydantic schemas, "data_class": "synthetic" on every response	Business logic (thin layer only)
Frontend (<code>web/</code>)	Canvas 2D dashboard; SYNTHETIC banner; JS ticker replay	Compute availability client-side (server is source of truth)
Audit (<code>core/audit.py</code>)	Hash-chained append-only log + verify function	Allow updates/deletes

API sketch (FastAPI)

GET <code>/api/fleet/status</code>	→ tails with MC/PMC/NMC, counts, data_class
GET <code>/api/fleet/forecast?days=7</code>	→ per-day MC rate, unfilled sorties
GET <code>/api/aircraft/{tail_no}</code>	→ tail detail, components, RULs, explanation
GET <code>/api/drivers</code>	→ ranked NMC drivers top-5
POST <code>/api/whatif</code>	→ {lever_json} → {before, after, tails_changed}
GET <code>/api/scenarios</code>	→ stored scenario runs
POST <code>/api/scenarios/{id}/apply</code>	→ make a scenario the working plan
GET <code>/api/cannibalization/advice?recipient=</code>	→ donor proposal + net effect + guardrail checks
GET <code>/api/audit</code>	→ hash-chained log entries

Every response includes `"data_class": "synthetic"` . OpenAPI auto-generated at `/docs` for judges who ask.

Data flow (one sentence per hop)

Generator seeds the world → feature store builds identical train/inference frames → GBM learns RUL per component class with tail-split validation → aggregator turns RUL+spares+crew+demand into per-tail states and the 7-day MC forecast → what-if/driver/cannibalization modules read the same pure state → FastAPI serves it → Canvas dashboard renders it with the SYNTHETIC banner everywhere.